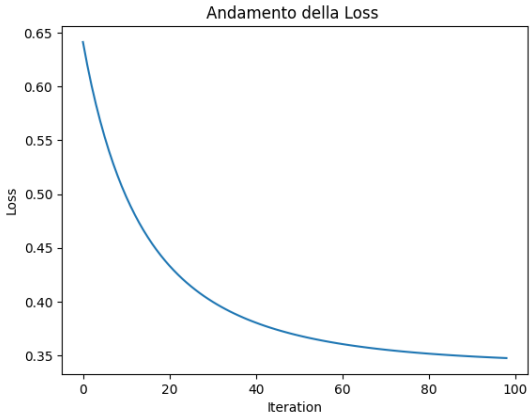
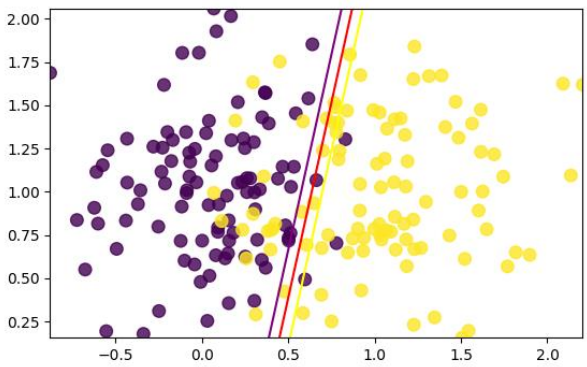
Zero Edoardo, SMID

Laboratorio 3 : Logistic Regression

Nel presente laboratorio sono state approfondite funzioni già trattate nei laboratori in precedenza , e in particolar modo si è studiato l'algoritmo della *Discesa del Gradiente* che rappresentauna tecnica fondamentale nella regressione logistica per trovare un'approssimazione della soluzione in quanto analiticamente risulta impossibile. In particolare sono state definite due funzioni : una per determinare il gamma ottimale (*learning* rate), chiamata **optimal_gd_learning_rate** e l'altra per stimare i pesi del classificatore utilizzando il *Training Set* **train_logreg_gd**.
La scelta del gamma ottimale è fondamentale nell'algoritmo del *GD* in quanto garantisce che la funzione *loss* decresca stabilmente e non troppo velocemente o lentamente.
Il criterio di arresto della funzione **train_logreg_gd** (*ciclo while*) prevede che le iterazioni si fermino se si raggiunge un numero massimo di iterazioni fissato (*maxiter*) oppure se la variazione della perdita tra due iterazioni successive è infinitesima quest'ultima condizione indicherebbe che l'algoritmo sta convergendo verso una soluzione.
E' importante considerare un numero sufficiente di iterazioni in modo tale che l'algoritmo possa trovare una soluzione ottimale e non si arresti prematuramente. Nella GD , la *loss* diminuisce in modo stabile ad ogni iterazione perché l'aggiornamento dei pesi è calcolato su tutti i campioni del dataset. Pertanto ,se la perdita diminuisce stabilmente, significa che l'algoritmo sta apprendendo correttamente.
Successivamente si è definita la funzione **predict_logreg** che restituisce il valore della predizione di Y e la probabilità associata alla previsione di ciascun punto , questo valore rappresenta la confidenza del modello nel classificare un punto come appartenente alla classe +1.
Nella fase iniziale dell'analisi sono stati generati il *Training Set* e il *Test Set* aggiungendo un bias . E' stata quindi eseguita una *regressione logistica* utilizzando le funzioni descritte precedentemente. E' stato tracciato il grafico dei valori della loss ad ogni iterazione dell'algoritmo della discesa del gradiente, si sono calcolati gli errori e si è visualizzata la frontiera di classificazione.





Come evidenzia il grafico si può notare che i valori della *loss* tendono a decrescere all'aumentare delle iterazioni, quindi la funzione **train_logreg_gd** è stata ben definita e il *GD* lavora correttamente.
La regressione logistica non solo fornisce la predizione della classe a cui appartiene un punto, restituisce anche la probabilità con la quale il modello compie tale stima. E' stata definita quindi una nuova funzione utile per visualizzare gli intervalli di confidenza con la quale il modello stima l'appartenenza dei punti alla classe 1. Si è definita una soglia di probabilità con la quale i punti appartengono alla classe 1 (*thresold*).

Osservando lo scatterplot:

- I punti che stanno sulla retta tracciata in rosso sono quelli predetti dal modello che hanno probabilità del 50% di appartenere alla classe 1, quindi quelli che il modello non è in grado di classificarli
- I punti che giacciono tra la retta rossa e quella gialla hanno una probabilità tra il 50/70% di appartenere alla classe 1, quelli a destra della gialla hanno prob. > 70% di appartenere alla classe 1.

- Infine quelli a sinistra della retta viola hanno probabilità < 30% di appartenere alla classe 1, quindi probabilità >70% di appartenere alla classe -1.

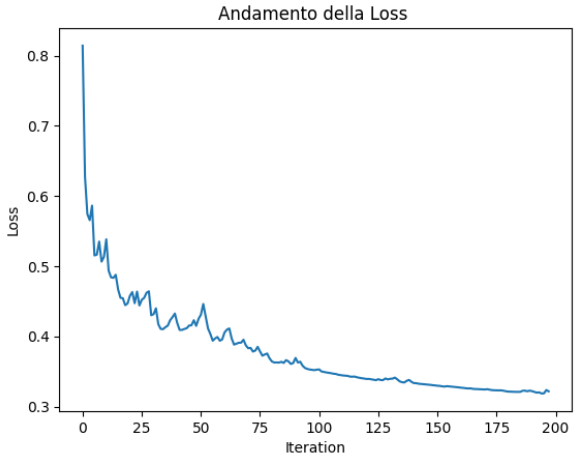
Successivamente si è definita una funzione che calcolava l'errore del modello solo sulle predizioni dei punti che avevano una percentuale >= 70% di appartenere a una classe (+- 1) e si è confrontato l'errore con quello della predizione su tutto il *Test Set* e si osserva che l'errore diminuisce.

Confronto GD e SGD

Nell'ultima parte di laboratorio è stato implementato l'algoritmo di *Discesa del Gradiente Stocastica* .
L'*SGD* riduce il numero di operazioni necessarie per aggiornare i pesi dell'algoritmo rispetto alla tecnica *GD*, perché, invece di calcolare il gradiente su tutto il dataset lo calcola su un singolo campione casuale, quindi dal punto di vista computazionale è migliore rispetto al *GD*.
Un'altra differenza sostanziale tra i due algoritmi è la scelta del *learning rate* , nel laboratorio sono stati proposti due valori differenti : 1 / sqrt(n) e 1 / sqrt(j).

- 1 / sqrt(n) : è fisso quindi il passo rimarrà costante per tutto il tempo, in particolare se *n* è grande il tasso rimarrà piccolo fin dall'inizio, ciò potrebbe rendere la convergenza abbastanza lenta.
- 1 / sqrt(j) : diminuisce con l'aumentare delle iterazioni che è una buona strategia perché inizialmente i passi sono più grandi poi diminuiscono sempre di più permettendo di stabilizzarsi più velocemente al minimo.

Il criterio d'arresto utilizzato per la *GD* non è direttamente adatto per la *SGD*.
Nella *GD*, la *loss* diminuisce in modo stabile a ogni iterazione. In *SGD* , invece, la *loss* può fluttuare in modo significativo tra un'iterazione e un'altra perché l'aggiornamento si basa su un singolo campione casuale per volta. Per questo motivo nella condizione di arresto dell'algoritmo si toglie la condizione :
abs(loss - loss_old) <= epsilon che potrebbe causare l'arresto dell'algoritmo prima del raggiungimento della soluzione ottimale. Quindi l'algoritmo si ferma solo dopo un numero prefissato di iterazioni (*maxiter*) perché garantisce che il modello abbia avuto abbastanza aggiornamenti sui pesi per convergere.



Confrontando questo grafico con quello della *GD*, si può notare che entrambi convergono a una soluzione ma l'andamento della funzione è diverso .
Nel caso del *GD* la discesa è regolare e monotona, portando a una convergenza più stabile e meno oscillante.
Nel caso della *SGD*, la curva mostra oscillazioni piuttosto marcate prima di stabilizzarsi.